

Arquitetura Concorrente em C++20 para Geração Procedural e Renderização de Malhas 3D

André Vitor Bastos de Macêdo
Instituto Federal de Educação, Ciência e Tecnologia Catarinense
Campus Blumenau
andre.macedo@estudantes.ifc.edu.br

Ricardo de la Rocha Ladeira
Instituto Federal de Educação, Ciência e Tecnologia Catarinense
Campus Blumenau
ricardo.ladeira@ifc.edu.br

Paulo Cesar Rodacki Gomes
Instituto Federal de Educação, Ciência e Tecnologia Catarinense
Campus Blumenau
paulo.gomes@ifc.edu.br

RESUMO

A geração procedural e a extração geométrica de malhas 3D (tridimensionais) densas impõem desafios arquiteturais críticos, frequentemente sobrecarregando a *thread* principal e degradando a fluidez da renderização gráfica. Este artigo apresenta uma arquitetura assíncrona baseada no padrão *Task Scheduler*, implementada em C++20, para gerenciar a geração e preparação concorrente de terrenos baseados em Ruído de Perlin. O estudo demonstra como o isolamento do contexto OpenGL e o uso de filas de prioridade multinível otimizam a aquisição de dados e mantêm a estabilidade do da taxa de quadros em simulações intensivas.

CCS Concepts:

- **Computing methodologies** → Rendering
- **Computing methodologies** → Concurrent computing methodologies

Keywords: Procedural Generation, C++20, Graphics Engine, Concurrent Programming, OpenGL

1 INTRODUÇÃO

Numa versão sequencial convencional, gerar terrenos de forma procedural e preparar malhas 3D complexas são tarefas executadas diretamente na *thread* principal, sobrecarregando-a quando ocorrem junto ao laço de renderização. Isso faz com que a taxa de quadros por segundo (FPS) caia drasticamente, podendo causar travamentos na aplicação.

Motores gráficos que usam a interface de programação (API, *Application Programming Interface*) OpenGL¹ sofrem ainda mais com esse problema. Por design, o contexto OpenGL é vinculado a uma única *thread*, exigindo que o laço de desenho e as alterações de estado dos gráficos aconteçam exclusivamente na *thread* principal [8]. Se essa mesma *thread* tiver que parar para calcular um mapa de ruído ou gerar a geometria da malha em uma abordagem sequencial, a renderização é interrompida. Portanto, é preciso isolar o processamento pesado para garantir que a interface continue responsiva.

Para resolver isso, este artigo apresenta uma abordagem paralela baseada no padrão *Task Scheduler* [9] desenvolvido com os recursos modernos do C++20. O sistema utiliza filas de prioridade multinível para organizar a criação dos mapas de altura via Ruído de Perlin e a extração das malhas em *threads* de segundo plano (trabalhadoras), deixando a *thread* principal livre apenas para as chamadas gráficas e delegação das tarefas.

A contribuição deste trabalho é demonstrar como essa arquitetura consegue separar a renderização da preparação dos dados de forma eficiente. Além disso, mostra-se como o uso de `std::jthread` e *smart pointers* facilita o gerenciamento de memória em sistemas paralelos, evitando erros críticos e garantindo que

o motor gráfico continue rodando de forma fluida mesmo durante simulações intensas.

2 FUNDAMENTAÇÃO TEÓRICA

Este estudo se baseia em conceitos fundamentais de computação gráfica, geração procedural e técnicas de concorrência em C++. A seguir, serão discutidos os principais tópicos relacionados a esses conceitos.

2.1 Geração Procedural de Terrenos

A geração procedural de mapas de altura (*heightmaps*) neste estudo utiliza o Ruído de Perlin [7], um algoritmo de ruído gradiente que gera padrões pseudoaleatórios suaves simulando relevos naturais. A complexidade da malha resultante é determinada pela sobreposição de múltiplas camadas de ruído (oitavas). Essa geometria serve como base de entrada para todas as avaliações de desempenho da arquitetura proposta.

2.2 Renderização e Responsividade

Motores gráficos interativos utilizam um laço de renderização (*render loop*) para processar eventos, atualizar estados e desenhar malhas continuamente em intervalos regulares. O processamento síncrono de tarefas pesadas (como a geração e triangulação de malhas) na *thread* principal causa latências que interrompem esse ciclo, resultando em quedas bruscas de desempenho (engasgos ou *stuttering*).

2.3 Monitor

O padrão Monitor é um mecanismo de sincronização de alto nível que garante exclusão mútua através de bloqueios (*mutexes*) e variáveis de condição [4]. Ao encapsular o estado compartilhado, o monitor coordena a execução concorrente e previne condições de corrida (*race conditions*) entre as rotinas assíncronas e a *thread* principal.

2.4 Agendamento de Tarefas (*Task Scheduler*)

Um *Task Scheduler* é um componente de software responsável por gerenciar a execução de tarefas concorrentes. Ele mantém uma fila de tarefas pendentes e um conjunto de *threads* (*thread pool*) que processam essas tarefas em paralelo [4]. O *Task Scheduler* é projetado para otimizar o uso dos recursos do sistema, garantindo que as tarefas pesadas de geração de terrenos sejam delegadas para *threads* trabalhadoras.

2.4.1 Filas multinível

Filas multinível são estruturas de dados que organizam elementos em diferentes níveis de importância. No contexto do agendamento de tarefas, essa estrutura permite que as *threads* tra-

¹<https://www.opengl.org/>

balhadoras priorizem a execução de rotinas críticas em detrimento de tarefas secundárias.

2.5 Concorrência e Segurança de Memória em C++20

A concorrência moderna em C++20 baseia-se em `std::jthread`, que gerencia o ciclo de vida das threads via RAII (Aquisição de Recurso é Inicialização), e `std::stop_token`, para interrupções cooperativas. A sincronização passiva de espera é realizada via `std::condition_variable_any`. Para garantir a segurança de memória contra condições de corrida e falhas de segmentação (SIGSEGV), é recomendado o uso de ponteiros brutos por referências e ponteiros inteligentes (`std::unique_ptr` e `std::shared_ptr`), automatizando o controle de tempo de vida dos recursos compartilhados.

3 METODOLOGIA

Para avaliar o impacto do processamento paralelo na geração das malhas, desenvolveu-se uma arquitetura baseada no padrão *Task Scheduler*. A metodologia adotada divide-se na implementação estrutural do escalonador, na instrumentação para coleta de dados de desempenho e na definição de cenários de teste isolados. Todas as implementações realizadas estão disponíveis no repositório público do projeto².

3.1 Ambiente de Teste e Ferramentas

O hardware de testes consiste em um processador Intel Core i5-1235U (12 threads, frequência máxima de 4.40 GHz), 16 GB de memória RAM e GPU integrada Intel Iris Xe Graphics. O sistema operacional base é o Arch Linux (Kernel 6.15.9).

O código-fonte em C++20 foi compilado via GCC com otimização de tempo de execução -O3 e diretrizes rigorosas (-Wall, -Wextra, -Wpedantic e -Werror). A renderização gráfica e o laço de eventos utilizam OpenGL 4.6 e GLFW 3.4, com matemática provida pela biblioteca GLM 1.0.3 e compilação estruturada via CMake.

3.2 Cenários de Teste

Para avaliar os algoritmos, estruturou-se quatro cenários que cruzam isolamento e concorrência:

- **Bench Sequential (BS):** Geração sequencial isolada (sem motor gráfico).
- **Bench Parallel (BP):** Geração concorrente isolada via TaskMaster.
- **Engine Sequential (ES):** Integração com motor gráfico rodando na thread principal, mensurando o impacto no FPS.
- **Engine Parallel (EP):** Geração assíncrona concorrente com envio de buffers para o motor.

Avaliou-se a **Escala** (malha de 100x100 a 1000x1000 vértices) e as **Oitavas** (1 a 10 camadas de ruído) com 100 amostras por nível de complexidade, resultando em 1000 amostras por avaliação (Escala ou Oitavas). A dimensão amostral atende ao Teorema do Limite Central, garantindo normalidade das médias para a aplicação dos testes paramétricos de ANOVA e Tukey [2].

3.3 Instrumentação e Coleta de Dados

Para a coleta e organização das métricas de desempenho, desenvolveu-se um módulo de instrumentação que armazena os dados em memória através da estrutura `std::map<std::string, std::map<std::string, std::vector<double>>>` e os exporta para arquivos CSV. O registro das métricas é feito apenas após o

término de cada cenário de teste, garantindo que o *overhead* de alocação da estrutura não interfira nas medições de latência do sistema.

3.3.1 Métricas de Desempenho e Speedup

Para avaliar a eficiência da concorrência, monitorou-se o **Tempo de Extração** (latência na geração do ruído e sua conversão em malha), a **Eficiência do Cache** (taxa de *cache misses* medida via `perf stat`) e a **Responsividade (FPS)** do laço de renderização. O ganho global de desempenho foi quantificado pelo *speedup* (S), definido pela razão entre o tempo de execução sequencial (T_s) e paralelo (T_p): $S = \frac{T_s}{T_p}$

3.4 Pipeline de Processamento Sequencial

No pipeline sequencial (modos BS e ES), todas as etapas de geração de ruído e triangulação ocorrem linearmente na *thread* principal. O processamento segue uma ordem rígida: para cada configuração de teste, o sistema gera o mapa de ruído e realiza a extração de geometria antes de avançar para o próximo quadro ou amostra. As baterias de testes foram organizadas em 10 níveis incrementais de complexidade (passos), variando a escala de 100x100 a 1000x1000 ou o número de oitavas de 1 a 10. Embora funcional para malhas pequenas, o custo acumulado desse processamento síncrono sob alta complexidade bloqueia a *thread* principal no modo ES, causando engasgos visíveis e quedas de FPS.

3.5 Arquitetura da Solução Concorrente

Para isolar o processamento pesado e manter a *thread* principal dedicada à renderização, desenvolveu-se uma arquitetura baseada no padrão *Task Scheduler*, cuja lógica central é encapsulada na classe TaskMaster. Este componente é responsável por gerenciar a fila de prioridades, administrar o pool de threads trabalhadoras e coordenar a execução concorrente de forma assíncrona.

3.5.1 Estrutura do TaskMaster



Figura 1: Diagrama de classes da arquitetura do TaskMaster

A implementação do TaskMaster utiliza um conjunto de três filas de prioridade, representadas pelo enum class Priority com os níveis High (0), Medium (1) e Low (2). Essas filas são armazenadas em um `std::array` de `std::queue`, permitindo o acesso direto de cada nível de importância. No contexto desta pesquisa, as tarefas de prioridade High compreendem a geração de mapas de altura (ruído), as tarefas Medium envolvem a extração geométrica e triangulação da malha 3D correspondente e as tarefas Low referem-se à gravação de logs e exportação dos dados estatísticos.

No construtor da classe, o número de threads trabalhadoras é determinado dinamicamente através de `std::thread::hardware_concurrency()`. Para evitar a saturação completa dos núcleos do processador e garantir que a *thread* principal (responsável pela renderização e interface) permaneça responsiva, o sistema reserva um núcleo, instanciando $N - 1$ threads trabalhadoras. Estas threads são implementadas como objetos

²<https://github.com/andrevbastos/pad>

`std::jthread`, aproveitando o comportamento RAII para garantir que sejam finalizadas corretamente na destruição do escalonador.

Cada thread trabalhadora executa um loop contínuo que aguarda por novas tarefas utilizando uma `std::condition_variable_any`. A lógica de seleção de tarefas prioriza sempre as filas de maior importância: o trabalhador verifica sequencialmente as filas High, Medium e Low, extraindo a primeira tarefa disponível na fila de maior prioridade encontrada. Isso garante que tarefas críticas, como a geração da malha visível, sejam processadas antes de tarefas de menor impacto, como a exportação de dados estatísticos.

O loop de execução é projetado para ser interrompido de forma cooperativa através do uso de `std::stop_token`, permitindo que as threads sejam encerradas de forma segura quando o escalonador for destruído ou quando uma interrupção for solicitada.

```
workers.emplace_back([this, id](std::stop_token st) {
    while (!st.stop_requested()) {
        // Lógica de espera e execução de tarefas
    }
});
```

Em cada iteração, a thread trabalhadora aguarda por uma notificação indicando que uma nova tarefa foi adicionada à fila. A espera é implementada utilizando `std::condition_variable_any`, que bloqueia a thread até que uma tarefa esteja disponível ou até que uma solicitação de parada seja feita.

```
// Bloco de código para uso do lock
{
    std::unique_lock<std::mutex> lock(mtx);
    auto stopCon = [this]{
        return
            !taskQueues[0].empty() ||
            !taskQueues[1].empty() ||
            !taskQueues[2].empty();
    };
    if (!cv.wait(lock, st, stopCon)) return;
```

Após ser notificada, a thread trabalhadora verifica as filas de tarefas em ordem de prioridade. A primeira tarefa disponível na fila de maior prioridade é extraída e executada. Se nenhuma tarefa estiver disponível, a thread retorna ao estado de espera. Mesmo que um lock tenha sido adquirido, a função `wait` do `std::condition_variable_any` é projetada para liberar o lock enquanto a thread está bloqueada, permitindo que outras threads adicionem tarefas à fila. Quando a thread é acordada, o lock é automaticamente re-adquirido e destruído com o escopo do bloco.

```
for (int q = 0; q < 3; ++q) {
    if (!taskQueues[q].empty()) {
        task = std::move(taskQueues[q].front());
        taskQueues[q].pop();
        break;
    }
}
// Fim do bloco de código para uso do lock
}

if (task) task(st);
```

O método `addTask` é o ponto de entrada para submissão de tarefas. Através de modelos de programação (*templates*) e metaprogramação em tempo de compilação, ele aceita funções que requerem ou não um `std::stop_token`. Se a função fornecida não aceita o token, ela é encapsulada em uma função lambda

que o ignora. O método adquire exclusão mútua via `std::mutex` para inserir a tarefa na fila correta e, em seguida, notifica a `std::condition_variable_any` para acordar uma thread trabalhadora ociosa, prevenindo condições de corrida e garantindo processamento assíncrono imediato.

Por fim, o ciclo de vida do escalonador é encerrado de forma cooperativa através do seu destrutor (Listagem 3). O uso combinado de `notify_all` para acordar threads bloqueadas e `request_stop` do `std::jthread` garante um encerramento limpo e livre de *deadlocks*.

3.5.2 Paralelização da Aquisição de Dados

A paralelização da aquisição de dados divide-se conforme o cenário: no modo EP (Engine Parallel), uma thread trabalhadora gera as malhas e as insere em uma fila assíncrona protegida por `std::mutex` para consumo pelo laço principal da engine, eliminando travamentos gráficos. No modo BP (Bench Parallel), o TaskMaster distribui as repetições do benchmark entre as threads secundárias, utilizando `std::mutex` e variáveis de condição para sincronizar o encerramento do passo antes que a thread principal consolide os dados estatísticos.

3.6 Controle de Recursos do Sistema Operacional

Para garantir um ambiente quiescente e mitigar ruídos experimentais [3], o sistema operacional foi configurado para desativar serviços de segundo plano, operar em modo de console virtual (TTY [5]) e fixar a CPU sob a política de alto desempenho (*performance*). Adicionalmente, os experimentos de benchmark e de engine foram conduzidos de forma alternada ao longo de 5 execuções independentes, com intervalos de resfriamento de 10 segundos, assegurando a estabilização térmica do processador e a consistência das medições.

4 RESULTADOS E DISCUSSÃO

O desempenho dos algoritmos foi avaliado com base nas métricas de tempo de extração, eficiência do cache e responsividade (FPS). Os resultados obtidos demonstram uma melhoria significativa na maior parte dos cenários avaliados quando a arquitetura concorrente é utilizada, sobretudo em termos de vazão global (throughput) e estabilidade do FPS.

4.1 Tratamento de dados

Mesmo rodando o algoritmo de forma isolada em linha de comando, interrupções temporárias do sistema operacional, oscilações na frequência da CPU (thermal throttling) ou pequenos atrasos de alocação de memória podem gerar picos isolados de latência (outliers) [6]. Para tratar esses pontos de dados discrepantes de forma matematicamente rigorosa (como recomendado para relatórios acadêmicos), implementamos o método da Amplitude Interquartilica (IQR) [1]:

- **Deteção:** Para cada configuração de teste (mesma escala/oitava e mesmo modo), calculou-se a amplitude interquartilica. Valores de tempo fora do intervalo das amostras foram classificados como outliers.
- **Imputação de valores:** Os outliers detectados foram substituídos pela **mediana** do seu respectivo grupo. Isso preserva o tamanho amostral original ($N = 100$) e a tendência central, mas estabiliza o desvio padrão e o erro residual, garantindo que o modelo da ANOVA atenda ao pressuposto de homogeneidade de variância.

4.2 Comparação de benchmark

Para os testes de benchmark, as médias dos modos BS e BP foram comparadas estatisticamente através de ANOVA de dois fatores e teste pós-hoc de Tukey com nível de significância $\alpha = 0,05$, adotando-se a Escala e o Número de Oitavas como fatores independentes [2].

Os resultados detalhados dos benchmarks puros (BS e BP) em função da escala e do número de oitavas são apresentados na Tabela 1 e na Tabela 2. Observa-se que, para a geração de uma única malha isolada, o modo sequencial é cerca de duas vezes mais rápido que o paralelo em todos os cenários. Na escala de 100×100 vértices, o tempo médio foi de 25,41 ms (BS) contra 55,32 ms (BP) (speedup de 0,46x), enquanto na escala de 1000×1000 o tempo sequencial registrou 2.295,97 ms frente a 4.859,25 ms do paralelo (speedup de 0,47x). Esse comportamento se repete no teste de oitavas, com speedups variando de 0,44x (1 oitava) a 0,51x (10 oitavas).

Tabela 1: Benchmark do tempo de geração de malha em função da escala.

Escala	Seq. Médio (ms)	Par. Médio (ms)	Speedup
100x100	25,41 $\pm$ 0,05	55,32 $\pm$ 2,11	0,46x
200x200	98,52 $\pm$ 0,16	216,95 $\pm$ 8,85	0,45x
300x300	218,54 $\pm$ 0,18	477,07 $\pm$ 20,19	0,46x
400x400	380,11 $\pm$ 0,28	808,81 $\pm$ 32,80	0,47x
500x500	585,75 $\pm$ 0,33	1241,59 $\pm$ 51,58	0,47x
600x600	842,92 $\pm$ 0,41	1792,47 $\pm$ 75,83	0,47x
700x700	1132,59 $\pm$ 0,69	2396,33 $\pm$ 100,96	0,47x
800x800	1483,53 $\pm$ 0,87	3182,91 $\pm$ 137,72	0,47x
900x900	1871,30 $\pm$ 1,08	3967,01 $\pm$ 168,51	0,47x
1000x1000	2295,97 $\pm$ 0,78	4859,25 $\pm$ 207,72	0,47x

Tabela 2: Benchmark do tempo de geração de malha em função das oitavas de ruído.

Oitavas	Seq. Médio (ms)	Par. Médio (ms)	Speedup
1	131,01 $\pm$ 0,28	295,96 $\pm$ 10,75	0,44x
2	176,00 $\pm$ 0,12	387,60 $\pm$ 12,80	0,45x
3	222,53 $\pm$ 0,11	496,05 $\pm$ 19,30	0,45x
4	270,30 $\pm$ 0,17	586,36 $\pm$ 21,92	0,46x
5	320,57 $\pm$ 0,23	688,34 $\pm$ 27,81	0,47x
6	374,70 $\pm$ 0,27	808,35 $\pm$ 35,60	0,46x
7	433,60 $\pm$ 0,32	895,33 $\pm$ 40,17	0,48x
8	491,28 $\pm$ 0,34	995,85 $\pm$ 48,09	0,49x
9	548,77 $\pm$ 0,33	1089,94 $\pm$ 52,81	0,50x
10	605,42 $\pm$ 0,39	1192,29 $\pm$ 57,34	0,51x

A aparente contradição da versão paralela ser mais lenta para processar uma única malha (latência da tarefa) é explicada ao analisar o tempo total necessário para processar o lote completo de testes (vazão ou *throughput*). Enquanto o lote completo de testes (composto por um total de 400 malhas 3D, sendo 200 no teste de escala e 200 no de oitavas) no modo Sequencial levou 250,32 segundos para ser concluído, o modo Paralelo finalizou todo o trabalho em apenas 46,69 segundos — representando um **speedup** global de 5,35x.

Essa diferença de comportamento entre a latência unitária e a vazão global deve-se ao fato do TaskMaster distribuir as diferentes repetições do benchmark concorrentemente entre os núcleos físicos da CPU. Embora cada thread sofra com o *overhead* de organização e sincronização, a execução paralela de múltiplas tarefas independentes maximiza o uso do processador.

Fisicamente, a perda de desempenho individual nas execuções paralela é justificada pela disputa por recursos de memória. Os dados coletados utilizando contadores de hardware (perf) apontam que o modo Sequencial apresentou uma taxa de erro de cache

(*cache misses*) de 30,93%, enquanto o modo Paralelo subiu para 36,48%. A execução simultânea de múltiplas threads de geração de malha força a CPU a realizar acessos frequentes à memória RAM física. Isso resulta em um aumento significativo de *cache misses*, o que explica a queda de desempenho individual. Enquanto o ganho global de vazão é evidenciado pela redução drástica do tempo total necessário para processar o lote completo de malhas.

4.2.1 Variabilidade

A análise de variabilidade revela comportamentos opostos entre os modos. No modo Sequencial, a dispersão é mínima (desvio padrão de 3,99 ms na escala 1000×1000), gerando boxplots achatados e previsíveis devido à execução linear e isolada no núcleo 2. Em contrapartida, o modo Paralelo apresenta alta dispersão (desvio padrão de 1.059,81 ms para o mesmo cenário), gerando caixas amplas nos boxplots (Figura 2 e Figura 3). Essa instabilidade decorre da concorrência introduzida pelo sistema operacional, onde o agendamento dinâmico de threads do TaskMaster causa latências de barramento de memória, trocas de contexto (*context switching*) e disputa por locks de sincronização.

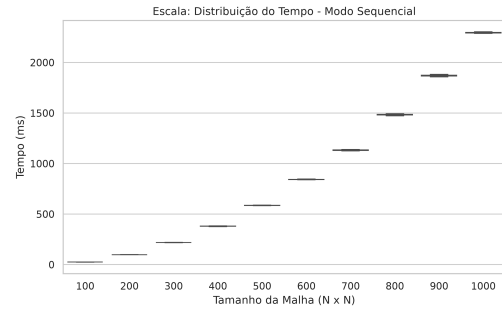


Figura 2: Dispersão do tempo de geração por escala no modo Sequencial.

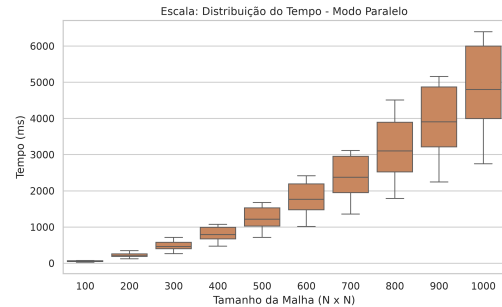


Figura 3: Dispersão do tempo de geração por escala no modo Paralelo.

4.2.2 Análise Estatística

Para avaliar de forma cientificamente se as diferenças observadas entre os tempos médios de geração dos modos Sequencial e Paralelo são estatisticamente significativas, realizou-se uma análise baseada em testes de hipóteses:

- **Hipótese Nula (H_0):** Não há diferença significativa nas médias dos tempos de geração entre os modos Sequencial e Paralelo para uma mesma configuração de parâmetros.
- **Hipótese Alternativa (H_1):** Há uma diferença estatisticamente significativa entre as médias de tempo de geração dos modos.

Primeiramente, aplicou-se a ANOVA de duas vias para avaliar a influência isolada do modo de execução (Sequencial ou Paralelo), do valor do parâmetro (Escala ou Oitavas) e sua interação [2]:

- **Experimento de Escala:** Revelou efeitos muito significativos para todos os fatores. O fator modo de execução obteve

p-valor $< 0,001$, o fator Escala registrou p-valor $< 0,001$ e a interação entre ambos alcançou p-valor $< 0,001$.

- **Experimento de Oitavas:** Também demonstrou significância estatística. O fator Modo registrou p-valor $< 0,001$, o fator Oitavas registrou p-valor $< 0,001$, enquanto o fator de interação obteve p-valor $< 0,001$.

A forte significância estatística da interação (p-valor $< 0,001$) em ambos os experimentos aponta que a diferença de desempenho entre os modos Sequencial e Paralelo depende diretamente do nível do parâmetro avaliado. Para isolar essas diferenças específicas em cada nível, aplicou-se o teste pós-hoc de Tukey [2].

No experimento de Escala, constatou-se que para grids pequenos de 100×100 (p-valor = 1,00) e 200×200 (p-valor = 0,79), **a diferença entre os modos não é estatisticamente significativa**. Nesses cenários, os dois algoritmos comportam-se de forma equivalente. Porém, a partir da escala 300×300 até a escala máxima de 1000×1000 , a hipótese nula H_0 foi consistentemente rejeitada (p-valor $< 0,05$), provando estatisticamente o atraso provocado pelo processamento paralelo de malhas individuais.

No experimento de Oitavas, a diferença foi significativa em todas as oitavas (de 1 a 10), com a rejeição da hipótese nula ocorrendo de forma estável (p-valor $< 0,001$) para todos os cenários de complexidade.

4.3 Desempenho no motor gráfico

Com a integração do TaskMaster ao motor gráfico, o impacto do processamento paralelo na fluidez da renderização foi avaliado através da métrica de FPS. Os resultados indicam que, mesmo com o aumento da latência individual para a geração de cada malha, a arquitetura concorrente permite que a thread principal mantenha uma taxa de quadros estável, evitando quedas bruscas de FPS e travamentos visíveis.

Nos testes a taxa de quadros por segundo foi limitada a 60 FPS para garantir uma experiência fluida. O modo Sequencial, ao bloquear a thread principal durante a geração da malha, resultou em quedas significativas de FPS, especialmente em configurações de alta complexidade (grids maiores e mais oitavas). Em contraste, o modo Paralelo conseguiu manter a taxa de quadros estável em 60 FPS, mesmo com o aumento da latência de geração, demonstrando a eficácia da arquitetura concorrente em isolar a thread de renderização das tarefas pesadas de processamento.

Os resultados obtidos no motor gráfico para as variações de escala e de oitavas estão consolidados na Tabela 3 e na Tabela 4. Fica evidente que, no modo Sequencial, o tempo de geração na thread principal degrada a renderização, derrubando o FPS de 39 (escala 100×100) para inviáveis 0,44 FPS (1000×1000 , com bloqueio de 2,29s) ou 1,65 FPS (10 oitavas). Em contrapartida, o modo Paralelo mantém a taxa estável no limite físico de 60 FPS em todas as configurações, delegando latências de até 3,89s (escala máxima) e 1.042,8 ms (oitavas máxima) a threads secundárias.

Tabela 3: Tempo de processamento e taxa de quadros (FPS) em função da escala na engine gráfica.

Escala	Seq. Tempo (ms)	Seq. FPS	Par. Tempo (ms)	Par. FPS
100x100	25,6	39	49,8	60
200x200	99,0	10	179,9	60
300x300	218,2	4,57	411,5	60
400x400	380,0	2,63	703,3	60
500x500	585,3	1,71	1071,0	60
600x600	842,4	1,19	1511,0	60
700x700	1131,7	0,88	2045,3	60
800x800	1481,8	0,67	2658,7	60
900x900	1871,7	0,53	3333,7	60
1000x1000	2295,0	0,44	3897,5	60

Tabela 4: Tempo de processamento e taxa de quadros (FPS) em função das oitavas de ruído na engine gráfica.

Oitavas	Seq. Tempo (ms)	Seq. FPS	Par. Tempo (ms)	Par. FPS
1	131,0	7,61	235,3	60
2	176,3	5,67	308,5	60
3	222,5	4,49	402,6	60
4	270,4	3,70	511,6	60
5	320,2	3,12	581,9	60
6	374,1	2,67	689,4	60
7	433,1	2,31	804,4	60
8	490,7	2,04	907,6	60
9	548,0	1,82	1010,8	60
10	605,4	1,65	1042,8	60

A dispersão do FPS obtido durante a simulação por escala pode ser observada na Figura 4 e na Figura 5. Os gráficos contrastam a instabilidade e a perda acentuada de FPS do modo sequencial sob cargas altas com a estabilidade do modo paralelo no limite físico do motor gráfico.

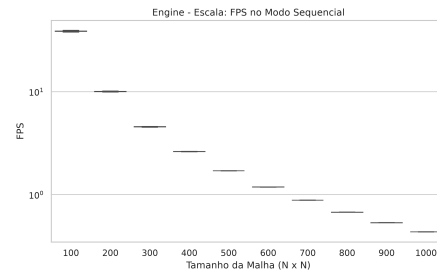


Figura 4: Dispersão da taxa de quadros (FPS) por escala no modo Sequencial na engine gráfica.

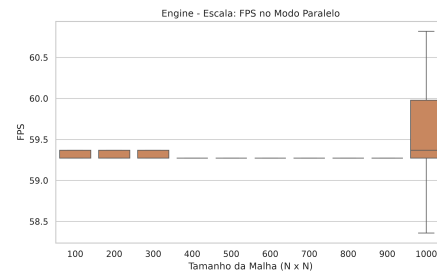


Figura 5: Dispersão da taxa de quadros (FPS) por escala no modo Paralelo na engine gráfica.

Cabe notar uma particularidade de visualização na Figura 5: embora os limites dos diagramas de caixa (*whiskers*) e do corpo da caixa aparentemente cobrir uma grande área da escala vertical, isso é

um artefato visual decorrente do ajuste automático de escala do eixo vertical no *software* de plotagem. Como a variação real do FPS no modo paralelo é quase nula (na ordem de 10^{-1} a 10^{-2} FPS), o eixo vertical foi ampliado em um intervalo microscópico.

Essa variação quase inexistente de FPS para a maioria das escalas ocorre porque a transferência de malhas pequenas para a GPU consome tempo desprezível da thread principal. Contudo, na escala máxima de 1000×1000 vértices, a malha possui cerca de um milhão de vértices. O envio desse grande volume de dados de vértices é processado obrigatoriamente pela thread principal de renderização. O *upload* desse buffer de dados no momento em que a malha fica pronta consome alguns milissegundos do tempo limite do quadro, explicando o desvio padrão de 0,56 FPS e as oscilações entre 57,8 e 60,8 FPS.

4.3.1 O Paradoxo da Responsividade

Embora o modo paralelo resulte em uma maior latência absoluta para concluir uma única tarefa (como visto anteriormente), a delegação desse processamento a threads secundárias pelo TaskMaster impede o bloqueio do laço de renderização principal. Assim, para aplicações gráficas interativas em tempo real, a estabilidade e a responsividade mostram-se mais importantes que o tempo bruto de execução do algoritmo de forma isolada.

4.3.2 Métricas de Cache na Engine

Os contadores físicos de CPU coletados via *perf* durante a execução junto ao motor gráfico corroboram as conclusões do benchmark isolado a respeito da disputa por recursos de memória. No modo Sequencial, a taxa de *cache misses* registrou 32,77%. Sob a execução do modo Paralelo, essa taxa subiu para 41,54%. Essa diferença reforça a explicação de que o processamento paralelo de múltiplas tarefas simultâneas aumenta significativamente a pressão sobre o subsistema de memória, resultando em um aumento substancial de *cache misses*. No entanto, mesmo com essa penalidade de desempenho individual, a arquitetura concorrente do TaskMaster permite que a aplicação mantenha uma experiência fluida e responsiva, além de alavancar o desempenho de gerações em massa.

4.3.3 Análise Estatística

Para consolidar as conclusões observadas no motor gráfico, aplicou-se a Análise de Variância (ANOVA) de duas vias sobre a taxa de quadros e o tempo de geração. A análise confirmou que o modo de execução possui efeito altamente significativo no tempo de processamento (p -valor $< 0,001$). O fator modo de execução (Sequencial ou Paralelo) também apresentou impacto estatístico massivo especificamente sobre a taxa de quadros p -valor $< 0,001$.

Por fim, o teste pós-hoc de Tukey corroborou que a melhoria de FPS obtida pela arquitetura concorrente é estatisticamente significativa em todas as escalas e oitavas avaliadas com p -valor $< 0,001$, validando cientificamente a eficácia da solução paralela.

5 CONCLUSÃO

Este trabalho apresentou uma arquitetura concorrente assíncrona baseada no padrão *Task Scheduler* e implementada em C++20 para solucionar o gargalo de processamento na geração procedural de terrenos e extração de malhas 3D integradas a motores gráficos. O objetivo principal foi garantir a estabilidade do FPS delegando tarefas intensivas a threads trabalhadoras secundárias.

Os resultados experimentais evidenciam que, embora a latência unitária tenha aumentado no modo paralelo devido à disputa de memória, o ganho de vazão alcançou um *speedup* de 5,35x em lote.

No motor gráfico, a arquitetura proposta sustentou a estabilidade em 60 FPS, enquanto o modo sequencial reduziu a renderização a 0,44 FPS sob alta complexidade. A eficácia da paralelização foi corroborada estatisticamente por ANOVA e teste de Tukey (p -valor $< 0,001$).

Do ponto de vista de engenharia de software, o uso dos recursos modernos do C++20 (como `std::jthread`, `std::stop_token` e ponteiros inteligentes) simplificou o gerenciamento do ciclo de vida das threads e garantiu a segurança de memória contra condições de corrida e vazamentos, reduzindo a complexidade do código.

Como trabalhos futuros, sugere-se a investigação de técnicas de transferência de dados mais eficientes para a GPU para mitigar o *overhead* observado no envio de buffers muito grandes da thread principal, utilizando instanceamento (*Instancing*). Adicionalmente, planeja-se estender essa arquitetura assíncrona para a paralelização de outros subsistemas do motor gráfico, como algoritmos de busca de caminho (*pathfinding*).

AGRADECIMENTOS

Os autores agradecem ao assistente de inteligência artificial Antigravity (desenvolvido pelo Google DeepMind) pelo auxílio na revisão textual e ortográfica, estruturação conceitual das ideias e na formatação das tabelas deste artigo. Ressalta-se que toda a concepção do estudo, implementação do software, execução dos experimentos e análise científica contidas neste trabalho são de inteira responsabilidade dos autores.

REFERÊNCIAS

- [1] Wilton de O. Bussab e Pedro A. Morettin. 2017. *Estatística Básica* (9ª ed.). Saraiva, São Paulo.
- [2] Jurandir Junior Domingues. 2026. Material didático.
- [3] Raj Jain. 1991. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley & Sons, New York.
- [4] Ricardo de la Rocha Ladeira. 2026. Material didático.
- [5] David J. Lilja. 2000. *Measuring Computer Performance: A Practitioner's Guide*. Cambridge University Press, Cambridge.
- [6] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, e Peter F. Sweeney. 2009. Producing wrong data without doing anything obviously wrong!. Em *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*, 2009. ACM, 265–276. <https://doi.org/10.1145/1508244.1508275>
- [7] Ken Perlin. 1985. An Image Synthesizer. Em *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, 1985. Association for Computing Machinery, New York, NY, USA, 287–296. <https://doi.org/10.1145/325334.325247>
- [8] Joey de Vries. 2014. Learn OpenGL: Enjoy learning modern OpenGL. Obtido 18 de abril de 2026 de <https://learnopengl.com/>
- [9] Anthony Williams. 2019. *C++ Concurrency in Action* (2nd ed.). Manning Publications.